By Satvik Pandey

Approach Document
SHL Conversational Assessment Recommender

This document summarises my implementation of a stateless FastAPI conversational recommender for SHL Individual Test Solutions. The service accepts full chat history per request and returns a grounded shortlist (1-10 assessments) with validated catalog URLs only.

1) Design choices

The key decision was to use a retrieval-grounded conversational pipeline instead of direct prompting over the full catalog. User intent is often incomplete in turn 1, so the agent must clarify before recommending. I added explicit policy guards for scope control (legal/refusal) and end-of-conversation handling to keep responses consistent under an 8-turn budget.

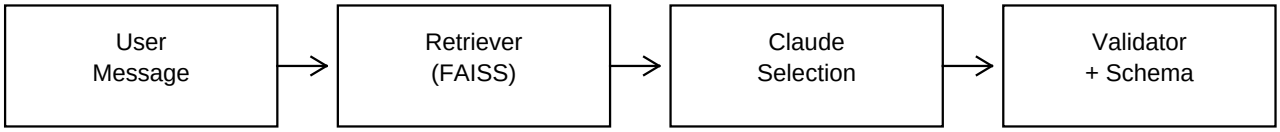
2) Catalog preparation

The raw shl_product_catalog.json contains malformed embedded newlines in string values, so a pre-parse sanitiser was required. I filtered out pre-packaged bundles (* Solution, * Short Form, Job Focused Assessment) to enforce assignment scope. The final catalog has 370 individual tests. test_type is derived deterministically from catalog keys to guarantee stable API formatting.

3) Retrieval setup

I used sentence-transformers/all-MiniLM-L6-v2 + FAISS inner-product search, with keyword overlap boosting and query expansion (e.g., leadership, sales, Java, contact center). This narrows each turn to top-25 candidates, reducing latency and hallucination risk. To fit Render free-tier memory, the index and model cache are pre-built during Docker build and loaded at runtime.

System flow (runtime request path)



Only top candidates are passed to the LLM; every returned URL is whitelisted against the local catalog.

4) Prompt design and policy orchestration

The system prompt forces JSON output and catalog grounding. I run rule checks before the LLM call: (a) vague first-turn -> one clarifying question, (b) legal/compliance -> refusal, (c) compare mode uses catalog descriptions only. Claude Sonnet then selects URLs from candidate items, and a hard validator drops any URL outside the whitelist. This provides deterministic safety on top of non-deterministic generation.

5) Evaluation method

I parsed gold URLs from C1-C10 and replayed full multi-turn traces against the live endpoint using eval/replay.py. Primary metric: Recall@10 on final shortlist. Secondary checks: schema compliance, catalog-only URL grounding, vague-turn clarification, and legal refusal behavior. Live benchmark: mean Recall@10 = 0.946 on C1-C10 with warm chat latency typically under 15 seconds.

By Satvik Pandey

Approach Summary (Page 2)

6) What did not work

Initial deployment failed on Render free tier due to memory pressure while loading sentence-transformers and building FAISS at startup. Also, relying purely on semantic retrieval produced lower recall on role-specific examples (e.g., graduate battery and full-stack refinement cases).

7) How I measured and improved quality

Improvements were made iteratively using replay traces and delta checks:
- Added deterministic guardrails (clarify/refuse/compare handling).
- Added targeted query expansion and domain-aware recommendation rules.
- Enforced strict URL whitelist and recommendation cap.
- Moved index/model build to Docker build stage and loaded cached artifacts at runtime.

Each change was verified with the same C1-C10 harness to confirm recall and behavior gains.

8) Deployment choices

Production uses Docker on Render with CPU-only PyTorch and pre-baked cache artifacts. This avoids runtime OOM and keeps startup predictable. Endpoints are intentionally minimal: /health and /chat only. A scheduled keep-warm job reduces free-tier cold-start impact during evaluation windows.

9) Stack and tools used

Python, FastAPI, Anthropic Claude Sonnet (claude-sonnet-4-20250514), sentence-transformers, FAISS, and custom evaluation harness scripts. Development iteration used AI-assisted coding support while keeping architecture, evaluation design, and final verification manual and test-driven.

10) Closing note

The final system is grounded, stateless, and aligned with assignment constraints: schema-safe responses, catalog-only recommendations, multi-turn refinement, and measurable evaluation quality. The deployed API is production-ready for SHL's automated replay checks.